Lesson: Mastering jQuery

May 15, 2025

Let's Get to Know Each Other

- “I’m Gehad Samy, Senior Software Engineer at SETS Intl.”
- Introduce yourself and a neighbor!

- Tell us your name, role,why you’re here and *One fun fact about you!*

Welcome & Agenda

Agenda:

- **First 3 Hours:** Theory & Concepts
  - Introduction to jQuery
  - Core Syntax & Selectors
  - DOM Manipulation
  - Events Handling
  - AJAX & Effects
- **Next 3 Hours:** Practical Lab
  - Guided exercises
  - Mini project implementation
  - Debugging and Q&A

What is jQuery?

- A fast, small, and feature-rich JavaScript library.
- Simplifies tasks like HTML document traversal, event handling, and AJAX.
- Developed in 2006 by John Resig.
- Once the most popular JS library before the rise of modern frameworks.
- Plays a major role in legacy systems and simpler frontend setups.
- CMS plugins (WordPress)

Why Use jQuery?

- Easy syntax (`$("#id")`, `$(".class")`, etc.)
- Cross-browser compatibility without writing polyfills.
- Powerful chaining for cleaner, fluent code.
- Rich plugin ecosystem for features like carousels, modals, and form validation.
- **Still useful in:**
  - Legacy enterprise applications
  - Quick prototypes
  - Lightweight projects

Including jQuery

- **Option 1:** Use a CDN

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

- **Option 2:** Host it locally

```
<script src="/js/jquery.min.js"></script>
```

- Ensure your code runs after the DOM loads:

```
$(document).ready(function() {
  // your code here
});
```

Selectors & Chaining

What Are jQuery Selectors?

- jQuery selectors are used to "find" or select HTML elements based on their name, id, classes, types, attributes, values, and more.
- They are modeled after CSS selectors but allow for richer querying and interaction.

Common Selector Types:

- **ID Selector:** Targets an element with a specific ID.

```
$("#header") // selects element with id="header"
```

- **Class Selector:** Targets all elements with a specific class.

```
$(".highlight") // selects all elements with class="highlight"
```

- **Tag Selector:** Targets all elements of a particular type.

```
$("p") // selects all <p> tags
```

- **Compound/Filtered Selectors:** Target nested or specific elements.

```
$("ul li:first") // first <li> in every <ul>
$("div:visible") // all visible <div>s
```

What is Chaining?

- Chaining means calling multiple jQuery methods on the same selector in a single statement.
- This leads to cleaner, more readable, and performant code.

Chaining Example:

```
$("#notice")
  .addClass("active")
  .css("color", "green")
  .fadeIn(300);
```

Traversing the DOM

What is DOM Traversal in jQuery?

- jQuery offers methods to navigate the DOM tree—moving up, down, and sideways through elements.
- These methods are useful for dynamically interacting with related elements without relying on complex selectors.
- Traversing enables modular code by finding context-specific elements like siblings, parents, or children.

What to Demonstrate:

- How to traverse from one element to its neighbor
- Looping through all `li` elements
- Real-time feedback in the console

Traverse the DOM tree to move between related elements.

Common Selector Types:

- **Parent Element:**

```
$(".item").parent();
```

- **Child Elements:**

```
$(".container").children();
```

- **Siblings:**

```
$(".selected").next(); // or .prev()
```

- **Find Descendants:**

```
$("#nav").find("li");
```

- **Looping with .each():**

```
$(".list li").each(function(index, element) {
  console.log(index, $(element).text());
});
```

DOM Manipulation

What is DOM Manipulation?

- DOM manipulation is the process of changing the document structure, content, and styles using JavaScript—in our case, with jQuery.
- jQuery simplifies this with methods to edit text, HTML, values, styles, and element positions.
- This is one of jQuery's most powerful features, making it easy to create dynamic web applications.

What to Demonstrate:

- Changing text and input values
- Adding new elements
- Updating DOM structure interactively

Modify content, structure, and styling of HTML elements using jQuery.

- **Text & HTML Content:**

```
$("#title").text("Hello");      // Plain text
$("#title").html("<b>Hello</b>"); // HTML content
```

- **Form Values:**

```
$("#username").val();           // Get value
$("#username").val("Gehad");    // Set value
```

- **Attributes and Props:**

```
$("img").attr("src", "image.jpg");
$("input").prop("checked", true);
```

- **DOM Structure:**

```
$("ul").append("<li>Item</li>");
$("ul").prepend("<li>First</li>");
$("li:last").remove();
$(".container").empty(); // remove inner HTML
```

- **Styling**

```
$("#box").css("background-color", "blue");
$("#box").addClass("highlight");
$("#box").toggleClass("hidden");
```

Events Handling

What is Event Handling in jQuery?

- Event handling allows your app to react to user actions such as clicks, key presses, form submissions, or mouse movements.
- jQuery simplifies this with intuitive .on() syntax for binding and delegation.
- It also provides shorthand methods like .click() or .hover() for basic interactions.

What to Demonstrate:

- Event binding
- Dynamic list creation
- Delegated event handling for newly added items

Listen and respond to user interactions.

- **Basic Event Binding:**

```
$("#btn").click(function() {
  alert("Clicked!");
});
```

- **Hover & Focus:**

```
$("#box").hover(
  function() { $(this).addClass("hovered"); },
  function() { $(this).removeClass("hovered"); }
);
```

- **Form Events:**

```
$("form").submit(function(e) {
  e.preventDefault();
  alert("Form submitted!");
});
```

- **Delegated Events:**

```
$("#list").on("click", "li", function() {
  $(this).toggleClass("done");
});
```

Effects & Animations

What Are Effects in jQuery?

- jQuery offers built-in animation and visual effects like hiding, showing, sliding, and fading elements.
- These can enhance UX by adding responsiveness and clarity.
- You can also create custom animations using `.animate()` and chain them with `.delay()` or `.queue()`.

What to Demonstrate:

- Use of .fadeToggle() and .animate()
- Combining effects
- Responsive feedback to user actions

jQuery provides powerful visual effects with minimal code.

- **Basic Visibility Controls:**

```
$("#panel").show();
$("#panel").hide();
$("#panel").toggle();
```

- **Fade In/Out:**

```
$("#alert").fadeIn(500);
$("#alert").fadeOut(300);
```

- **Slide Effects:**

```
$("#menu").slideDown();
$("#menu").slideUp();
```

- **Custom Animations:**

```
$("#box").animate({
  left: "+=100",
  opacity: 0.5
}, 800);
```

Queueing Effects:

```
$("#popup")
  .fadeIn(200)
  .delay(1000)
  .fadeOut(200);
```

Used to enhance UX with interactive transitions and visual feedback.

🔧 Other Useful Effects

Method	Description
.hide()	Instantly hides an element
.show()	Instantly shows an element
.fadeIn()	Fades element into view
.fadeOut()	Fades element out
.slideDown()	Slides element down into view
.slideUp()	Slides element up and hides it
.delay(ms)	Waits before starting the next effect
.queue()	Chains multiple effects with control

AJAX with jQuery

What is jQuery AJAX?

- AJAX (Asynchronous JavaScript and XML) allows you to fetch data from the server and update the page dynamically—without a full reload.
- jQuery wraps AJAX functionality into easy-to-use methods like `.get()`, `.post()`, and `.ajax()`.
- It's especially helpful when working with APIs or submitting forms in the background.

What to Demonstrate:

- Making GET requests to a public API
- Updating the UI with fetched data
- Working with asynchronous flow

Make asynchronous HTTP requests without reloading the page.

- **GET Request (fetch data):**

```
$.get("https://jsonplaceholder.typicode.com/users",
function(data) {
  console.log(data);
});
```

- **POST Request (send data):**

```
$.post("/api/submit", { name: "Gehad" },
function(response) {
  console.log("Submitted", response);
});
```

- **Generic AJAX Call:**

```
$.ajax({
  url: "/api/items",
  method: "GET",
  dataType: "json",
  success: function(result) {
    console.log(result);
  },
  error: function(xhr, status, error) {
    console.error("Error:", status);
  }
});
```

- **Loading Indicator Example:**

```
$("#loader").show();
$.get("/data", function(data) {
  $("#loader").hide();
  $("#output").html(data);
});
```

AJAX helps dynamically update parts of a web page without full reload.

jQuery Best Practices

1. **Wrap your logic with**

```
$(document).ready(function() {
  // All your jQuery code goes here
});
```

⚡ 2. Cache Repeated Selectors

- Instead of writing this repeatedly:

```
$("#myList").append(...);
$("#myList").css(...);
```

- Use a variable:

```
const $list = $("#myList");
$list.append(...);
$list.css(...);
```

✅ Improves performance and keeps code DRY.

📏 3. Keep Your Chains Readable

- This is fine:

```
$("#box").addClass("highlight").fadeIn(200);
```

- But this gets messy:

```
$("#box").addClass("a").css("b",
"c").fadeOut().delay(100).fadeIn().text("done").slideUp(100)
.slideDown(200);
```

✅ Break long chains or split into multiple steps if needed.

Break

Comprehensive Lab
jQuery in Action

Lab Objective

Apply the core jQuery concepts learned (selectors, DOM manipulation, event handling, AJAX) to build a small interactive application that integrates dynamic data and UI responsiveness.

Project Description: Build a basic client-side dashboard using jQuery that includes:

- A dynamic to-do list
- A live search feature powered by a public API (e.g., user data)
- Style and animation interactions

API Endpoint Used:

- GET `https://jsonplaceholder.typicode.com/users` → for live user data

Expected Deliverables

- A functional task list with:
  a. Add, remove, mark as complete
  b. Filter: All / Active / Completed
- A button to fetch and display users from API:
  a. Render as list/table
  b. Provide search box to filter by name or email
- All UI interactions styled using jQuery (fadeIn, slideToggle, etc.)

Instructions

- Create an HTML layout with the following components:
  - A form to add tasks
  - A section for filter buttons
  - A `div` or `ul` to display fetched users
- Use jQuery to:
  - Handle user input and bind events
  - Dynamically update and animate DOM elements
  - Call AJAX API and display data
- Bonus: Style the UI using `.addClass()` and `.toggleClass()`

Resources

1. Official jQuery Site
2. API Documentation
3. jQuery Cheat Sheet https://htmlcheatsheet.com/jquery/

Thank You

Thank you for participating!

Contact: gehadsamy96@gmail.com